

NexoCRM — Accounting-Readiness Change Document (Consolidated)

Branch: `develop-sreekesh` · **Scope:** all accounting-readiness commits (`14426be` , `ddb8df3` , `fb714cd`) · **Status:** merged to branch, tests green **Audience:** Business/QA stakeholders (Part A) + Frontend, Flutter and Backend engineers (Part B) **Purpose:** hand-off for testing and UI preparation before the Accounting module is built.

! READ THIS FIRST — the one rule that governs everything below

These changes **prepare** the CRM for a future Accounting module. They ship the database columns, API fields, validation rules and internal "hook" points the module will plug into — **but the Accounting module itself does not exist yet.**

There are therefore **two states** for every change in this document:

	LIVE NOW (accounting OFF — today)	DORMANT (activates when accounting is switched on)
New request fields	Accepted and stored (safe to start sending)	Consumed by accounting
New response fields	Present but empty / null	Populated with real data
Guard rules (lock/block)	Present but never trigger (nothing sets the trigger column)	Actively block edits/deletes
409 settlement rejection	Cannot happen	Can happen on mark-paid
Hooks / events	Fire but do nothing (no listeners)	Post journals / receipts

For QA: test that the CRM **behaves exactly as before** and the new fields are **present-but-null / accepted-but-inert**. You are testing that we added surface area **without changing existing behavior**. The "when-active" columns are documented so UI can be built ahead of time, not so they can be tested today.

PART A — For Business & QA Stakeholders (plain English)

We added the "plumbing" for accounting. Nothing customers do today changes. Here is what was added, area by area.

1. Quotes now carry tax information

Quote lines can now record **HSN/SAC code**, **GST rate %**, **line discount**, and a link to the **catalog product**. A quote can say whether its **amounts are tax-inclusive or tax-exclusive**, its **place-of-supply state**, and a **contract reference**. When a quote is accepted, these values are **frozen** (snapshot) so later catalog edits don't rewrite old quotes. *Tables:* `quotations_quotation` , `quotations_quotationlineitem` .

2. Payment plans got richer (still just a tracker, not a tax invoice)

What we used to loosely call an "invoice" is really a **payment-tracking plan**. It now records how bills should be raised (**billing mode**, **auto-issue**), whether amounts are tax-inclusive/exclusive, and links to a **project**. Each payment installment can record **TDS**, **GST-TDS/TCS**, **bank charges**, **bank/gateway reference**, **service period** (for subscriptions), and whether an advance is an **advance vs customer credit**. *Tables:* `quotations_payment` , `quotations_paymentrecord` .

3. Support tickets can be marked billable

A helpdesk ticket can be flagged **billable**, with a **billable amount** and/or a **billable product**. (Who ultimately gets charged is handled by accounting later.) *Table:* `crm_issue` .

4. Products carry tax defaults

Products/services now carry **nature of supply (Goods/Service)** and keep their **HSN/SAC** and **default GST rate**. The old **12% and 28% GST slabs** are flagged as **deprecated** (GST 2.0). *Table:* `crm_product` .

5. Projects can link to a quote and a cost centre

A project can be linked to the **quote** it came from and to a **cost-centre**. The project's money columns (costing, margin, etc.) are marked **deprecated** — accounting will become the source of truth for those numbers. *Table:* `projects_project` .

6. Company GST profile + India state list

An organization now stores its **GST state code**. A new read-only list of the **38 India GST state/UT codes** is available for dropdowns. The old manual **invoice prefix / next-invoice-number** fields are **deprecated** (accounting will own numbering). *Table:* `organizations_organization`.

7. New user roles & permissions for accounting

Two new job roles — **Accountant** and **Finance Viewer** — and seven accounting permission areas (setup, documents, purchases, settlements, periods, journals, reports) were added. They are gated behind a new **"Accounting" plan feature**, so only plans that include it will see accounting.

8. A GST Tax-Invoice print template

A GST-compliant **"Tax Invoice"** print layout was added alongside the existing Classic/Modern quote templates. It only fills in tax details once accounting is live.

9. Cleanups that protect data

Quote numbers are now **guaranteed unique** per company. Certain destructive actions (deleting/reverting a plan that has been billed by accounting) are **blocked** to protect financial history — but only once accounting is actually issuing documents.

PART B — For Frontend / Flutter / Backend Engineers

No new CRUD routes were added except one read-only lookup (states). Everything else is **new fields and new rules on existing endpoints**. All routes are under `/api/v1/`.

B0. How to read this section

- **Req** = you may send it in the request body. **Resp** = it appears in the response.
- **Live now** = active with accounting off. **When active** = behavior once accounting ships.
- Unless stated, new response fields return `null` / `false` / empty today.

B1. NEW endpoint — India GST state codes (lookup)

```
GET /api/v1/organizations/states/
```

- **Auth:** standard JWT. Read-only, static (38 rows), safe to cache client-side.
- **Response:** `[{ "code": "32", "name": "Kerala", "alpha": "KL" }, ...]`
- **Use for:** the **Place of Supply** and **Company State** dropdowns. Store the 2-char numeric `code` (that is what `place_of_supply_state_code` / `Organization.state_code` expect).

B2. Quotations — `/api/v1/quotations/quotations/`

Quote header — new fields

Field	Req/Resp	Type	Live now	Notes
<code>amounts_are</code>	Req+Resp	<code>"EXCLUSIVE"</code> / <code>"INCLUSIVE"</code>	✅ stored	Tax-exclusive vs inclusive. Frozen at acceptance.
<code>place_of_supply_state_code</code>	Req+Resp	2-char string	✅ stored	From the states lookup.
<code>contract_ref</code>	Req+Resp	string	✅ stored	Free-text contract reference.

Quote line items (`line_items[]`) — new fields

Field	Req/Resp	Type	Live now	Notes
<code>product_id</code>	Req (write)	UUID	✅	Links a line to a catalog product; snapshots its tax fields onto the line. Org-scoped — a foreign-org <code>product_id</code> is rejected.

Field	Req/Resp	Type	Live now	Notes
product	Resp (read)	UUID	✓	Read-back of the linked product.
hsn_sac	Resp (read-only)	string	✓ auto	Snapshoted from product; not client-writable.
rate_pct	Resp (read-only)	decimal	✓ auto	GST rate snapshot.
supply_nature	Resp (read-only)	"GOODS" / "SERVICE"	✓ auto	Snapshot.
line_discount	Req+Resp	decimal	✓	Per-line discount; line total is clamped ≥ 0 .
line_no	Resp	int	✓	Stable ordering.
source_package	Resp	UUID	✓	Set when a line was exploded from a package product.
quantity	Req+Resp	decimal	✓	Now supports fractional quantity.

Workflow to test (live now): - Sending a **package** `product_id` explodes it into component + adjustment lines (`source_package` set); the sum matches the catalog package total; no negative lines. - Sending a **cross-org** `product_id` is rejected (validation error, no line created). - Quote numbers are **unique per org** — a duplicate `quotation_number` is rejected by the DB.

B3. Payment plan — `/api/v1/quotations/payments/`

i Terminology: this endpoint is the **payment-tracking plan**, *not* a tax invoice. The response still exposes a **deprecated** `invoice_id` (see B8).

Plan (`Payment`) — new fields

Field	Req/Resp	Type	Live now	Notes
billing_mode	Req+Resp	INVOICE_ON_DUE / INVOICE_ON_COMPLETION / PERIODIC / MANUAL	✓	How bills are raised.
auto_issue	Req+Resp	bool	✓ stored (inert)	Auto-issue documents once accounting is on.
amounts_are	Resp (read-only)	EXCLUSIVE / INCLUSIVE	✓	Copied from the quote at acceptance, then read-only.
project	Req+Resp	UUID	✓	Optional link to a project.
contract_ref	Req+Resp	string	✓	

Payment record (`PaymentRecord`) — new fields

Field	Req/Resp	Type	Live now	Notes
tds_deducted	Req+Resp	decimal ≥ 0	✓	Only accepted when the record is paid ; counts toward plan completion.
gst_tds_deducted	Req+Resp	decimal ≥ 0	✓	
gst_tcs_deducted	Req+Resp	decimal ≥ 0	✓	
bank_charge	Req+Resp	decimal ≥ 0	✓	Accepted only when paid.
bank_account_id	Req+Resp	UUID	✓	
gateway_ref	Req+Resp	string	✓	
treat_as	Req+Resp	ADVANCE / CUSTOMER_CREDIT	✓	Advance treatment.
service_period_start / service_period_end	Req+Resp	date	✓	Subscription period.
milestone_task	Req+Resp	UUID	✓	Must be an <code>is_milestone</code> task in the plan's project (validated).
linked_document_id		UUID		Set by accounting when a tax document is issued.

Field	Req/Resp	Type	Live now	Notes
	Resp (read-only)		null today	
posted_receipt_id	Resp (read-only)	UUID	null today	Set by accounting when a receipt is posted.
record_summary	Resp (read-only)	object / null	null today	Accounting-supplied summary (invoice no., open balance...). Always present as a key.

Settlement math to test (live now): - A record marked **paid** with `amount_paid` + `tds_deducted` counts the **TDS toward completion** (cash figure and settled figure differ in the API). - Ten instalments of gross receipts do **not** auto-cancel the last one; plan completes only when all are settled. - Subscription plan total stays **per-period**; settling one period advances to the next.

B4. Settlement lifecycle & the 409 you must handle

When a payment record is created / marked-paid / un-paid / edited / deleted, the backend fires internal accounting hooks **inside the same DB transaction. Today there are no listeners, so nothing changes.** When accounting is active, a hook may **veto** the write (e.g. a closed accounting period), which rolls the CRM write back and returns:

```
HTTP 409 Conflict
{
  "code": "ACCOUNTING_POSTING_REJECTED",
  "cause": { ... },           // structured context (e.g. {"period": "2026-04"})
  "detail": "human-readable reason",
  "rule": "R7",              // optional: the accounting rule id that failed
  "next": "reopen_period"    // optional: hint for what the user can do
}
```

- **Frontend/Flutter:** add a handler for **409** with `code == "ACCOUNTING_POSTING_REJECTED"` on payment-record save/delete. Show `detail`; use `cause` / `rule` / `next` for a richer message. This is the **only** endpoint family that can return this envelope, and only when accounting is on.
- This is a new **structured error shape** (`{code, cause, detail, rule?, next?}`) distinct from the usual `{code, message}` / `{code, message, errors}`. All other errors are unchanged.

B5. Guard rules on destructive actions (dormant until accounting issues documents)

Once a payment record is **paid** or **linked to an issued tax document** (`linked_document_id` set by accounting), these are blocked with a **400** validation error: - **Deleting** the plan / its payment records. - **Reverting** the quote out of an accepted status (`status_id` change rejected). - **Rolling back** a converted customer to a lead (even with `force=true`). - **Editing** the plan's monetary/config + accounting-header fields (total, line items, payment type, currency, installment count/period, `amounts_are`, place-of-supply, `contract_ref`) — they **lock**.

Live now: `linked_document_id` is never set (accounting is off) and no record is auto-locked, so these guards never fire — existing delete/revert/rollback flows behave exactly as before. Build the UI to **surface the block reason** (the `detail` string) for when accounting is on.

B6. Helpdesk issues — /api/v1/crm/issues/

Field	Req/Resp	Type	Live now	Notes
billable	Req+Resp	bool	✓	Flag a ticket billable.
billable_amount	Req+Resp	decimal	✓	
billable_product_id	Req (write-only)	UUID	✓	Org-scoped ; foreign-org id rejected.
billable_product	Resp (read-only)	UUID	✓	Read-back.
billed_document_line_id	Resp (read-only)	UUID	null today	Set by accounting when the charge is posted.

Validation to test: when `billable = true`, the resulting row must carry **at least one of** `billable_amount` / `billable_product` — otherwise **400**.

B7. Products — `/api/v1/crm/products/`

Field	Req/Resp	Type	Live now	Notes
<code>supply_nature</code>	Req+Resp	GOODS / SERVICE	✓	Nature of supply.
<code>hsn_sac</code>	Req+Resp	string	✓	Validated format.
<code>tax_rate</code>	Req+Resp	decimal	✓	UI default only — the <i>effective</i> GST rate is resolved by accounting's rate master later.
<code>deprecated_rate</code>	Resp (read-only)	string / absent	✓	Appears only when <code>tax_rate</code> is 12% or 28% (GST 2.0 deprecation nudge).

UI note: show a warning when a product uses a **12% or 28%** rate (read `deprecated_rate` from the response).

B8. Deprecations — plan your migration away from these

Field	Endpoint / model	Replacement	Still works?
<code>invoice_id</code>	Payment + PaymentRecord responses	<code>linked_document_id</code> / <code>record_summary</code> (once accounting ships)	✓ read-only alias for the plan id; removed one release after accounting ships
<code>invoice_prefix</code> , <code>next_invoice_number</code>	Organization	Accounting's own numbering	✓ still writable; do not build new features on it
<code>total_costing_amount</code> , <code>total_purchase_cost</code> , <code>total_sales_amount</code> , <code>total_billable_amount</code> , <code>total_billed_amount</code> , <code>total_consumed_material_cost</code> , <code>gross_margin</code> , <code>per_gross_margin</code>	Project	<code>financials</code> object (once accounting ships)	✓ read-only; will be derived from the ledger

`ProjectSerializer` also gains `financials` (Resp, null today) and a `quotation` link (Req+Resp, UUID; validated to be same-org and same-customer as the project).

B9. RBAC, roles & feature gating (Backend / Admin UI)

- **7 new module codenames** (permission areas): `acc_setup`, `acc_document`, `acc_purchase`, `acc_settlement`, `acc_period`, `acc_journal`, `acc_report` — bundled as an **"Accounting" module group** in the role-creation UI.
- **2 new seeded roles:** **Accountant**, **Finance Viewer** (auto-seeded for new orgs).
- **Plan feature gate:** a new **accounting** feature bundles those 7 modules. Accounting endpoints/permissions are only exposed to plans that include the feature.
- **Backfill commands (ops — run at accounting go-live, not now):**
 - `python manage.py backfill_accounting_permissions --create-roles [--all|--org <id>] [--dry-run]` — provisions the two roles + grants the `acc_*` permissions to existing orgs.
 - `python manage.py backfill_quote_templates --all [--dry-run]` — seeds the Tax-Invoice template to existing orgs.

B10. Print templates

- New **"Tax Invoice"** template (`category = "invoice"`), GST-compliant layout, seeded for new orgs; never the default quote template.
- The template list endpoint returns it **unless you filter**: `GET /api/v1/quotations/quotation-templates/?category=invoice` returns invoice templates; the **quote-templates grid should request quote templates only** (omit invoice category) so "Tax Invoice" does not appear in the quote picker until accounting is live.

B11. Reference master data (context, not API)

The user uploaded accounting reference data under `docs/` (GST rate masters for goods/services, RCM, blocked-ITC, TDS nature & rate matrix, chart of accounts, expense/revenue categories, depreciation classes, professional-tax slabs, document-series prefixes, entity-behaviour,

statutory params YAML). These are **inputs for the future accounting module**, not current API changes — no action for CRM teams beyond awareness.

Per-Team Hand-off Checklist

Frontend & Flutter (shared)

Test (regression — behavior must be unchanged today): - Existing quote / payment / issue / product / project screens work as before with the new null/empty fields present. - New request fields (SB2, B3, B6, B7) are **accepted and persisted**.

Build ahead (UI to add now): - State dropdowns via `GET /organizations/states/` for place-of-supply / company state. - Quote line editor: HSN/SAC (read-only snapshot), GST %, line discount, product link, fractional qty. - Quote header: tax-inclusive/exclusive toggle, place-of-supply, contract ref. - Payment record form: TDS / GST-TDS/TCS / bank charge / bank ref / treat-as / service period / milestone. - Issue form: billable + amount/product (with the "needs a basis" validation). - Product form: supply nature + `deprecated_rate` warning banner for 12%/28%. - **Error handling:** the `409 ACCOUNTING_POSTING_REJECTED` structured envelope (SB4) and the `400` guard-block reasons (SB5). - Quote-template picker: filter to quote templates so "Tax Invoice" stays hidden until accounting is on.

Backend

- Run migrations (0002–0005 quotations, 0015 crm, 0016–0017 organizations, 0004 projects, RBAC module_name alters). Re-run `seed_features` and `seed_status_types` on fresh DBs.
- Confirm the accounting hooks are **no-ops with no listeners** (API responses byte-identical to pre-change) — this is the core acceptance test.
- Do **not** run the `backfill * ops` commands until accounting go-live.
- When the accounting module is built, it registers: settlement hooks (`quotations.hooks`), the `record_summary / project financials / company_context` providers, and writes pointer columns **only** via the sanctioned services (`quotations.services.payment_records` , `crm.services.issue_billing`) — never direct column writes.

End of consolidated change document.